

TEMA 5

LAB PARALELISMO DE DATOS SIMD

TEMA 5

LAB PARALELISMO DE DATOS - SIMD

ARQUITECTURA DE COMPUTADORES

Dpto. Arquitectura de Computadores

Universidad de Málaga



CONTENIDOS

- Introducción
- Registros
- Convenio
- Tipo de datos
- Instrucciones
- Cuestiones a tener en cuenta

Transparencias basadas en el curso:

- Oscar Plata. Altas prestaciones para datos y ciberseguridad. Universidad de Málaga



INTRODUCCIÓN

TEMA 5: LAB PARALELISMO DE DATOS - SIMD



INTRODUCCIÓN

- El objetivo de esta práctica es poner en práctica el conocimiento adquirido sobre las extensiones vectoriales presentes en la arquitectura del procesador
- Veremos dos maneras de explotar el paralelismo SIMD
 - **Auto-vectorización:** será el compilador quien se encargue de explotar el paralelismo de datos
 - **Intrínsecas del compilador:** añadiendo instrucciones (intrínsecas) al código
- Utilizaremos un código que calcula el **producto de dos matrices** cuadradas
 - Aparece en numerosas aplicaciones de diferentes ámbitos (aplicaciones gráficas, de simulación, científicas, etc.)
 - posee una estructura fácilmente extrapolable a otros códigos
- Usaremos **lenguaje de programación C**
 - Ejecutaremos sobre un sistema Linux



INTRODUCCIÓN

- Los procesadores multicore modernos incluyen soporte hardware para explotar el paralelismo SIMD

○ Arquitectura x86 (Intel, AMD)

- MMX (MultiMedia eXtensions), 1997: 8 registros vectoriales de 64 bits que sólo contienen enteros
- SSE (Streaming SIMD extensions), 1999: 32 registros vectoriales de 128 bits: xmm0, xmm1, ..., xmm7, ..., xmm15, ..., xmm31
- SSE2 (2001), SSE3 (2004), SSE4 (2007)
- AVX (Advanced Vector eXtensions), 2011, AVX2 (2013): 16 vectores de datos de 256 bits: ymm0, ymm1, ..., ymm7, ..., ymm15
- AVX-512 (2015): 32 vectores de datos de 512 bits: zmm0, zmm1, ..., zmm31

○ Otras arquitecturas: NEON (ARM), AltiVec (IBM)



REGISTROS

TEMA 5: LAB PARALELISMO DE DATOS - SIMD



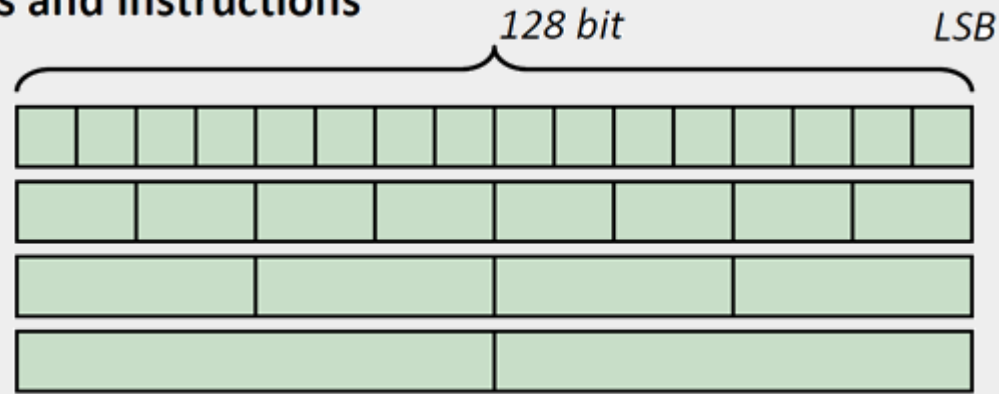
REGISTROS SSE3/SSE4

- 32 registers: xmm0, xmm1, ..., xmm7, ..., xmm15, ..., xmm31

Used for different data types and instructions

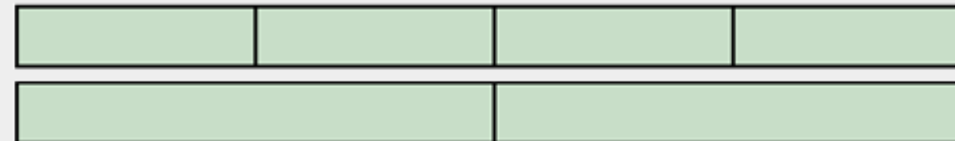
Integer vectors:

- 16-way byte
- 8-way 2 bytes
- 4-way 4 bytes
- 2-way 8 bytes



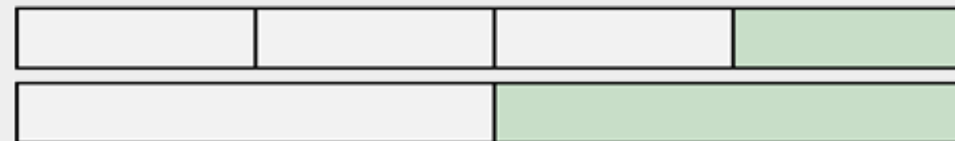
Floating point vectors:

- 4-way single (since SSE)
- 2-way double (since SSE2)



Floating point scalars:

- single (since SSE)
- double (since SSE2)



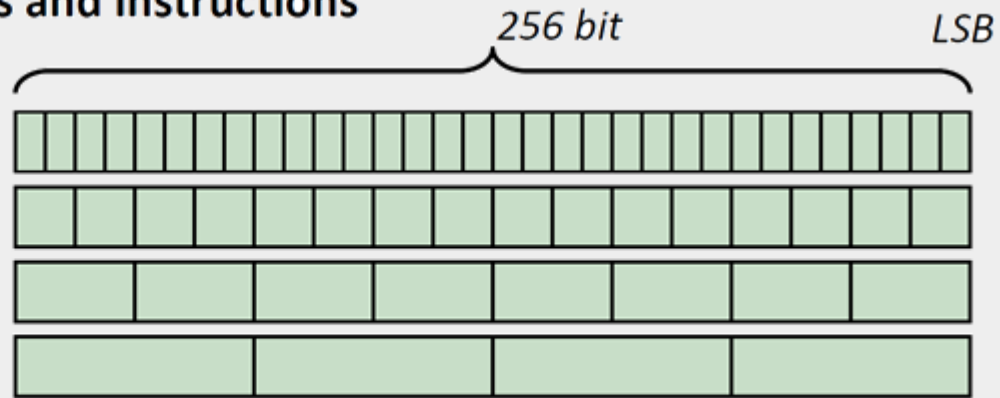
REGISTROS AVX

- 16 registers: ymm0, ymm1, ..., ymm7, ..., ymm15

Used for different data types and instructions

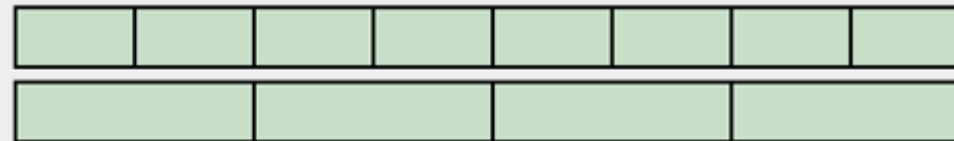
Integer vectors:

- 32-way byte
- 16-way 2 bytes
- 8-way 4 bytes
- 4-way 8 bytes



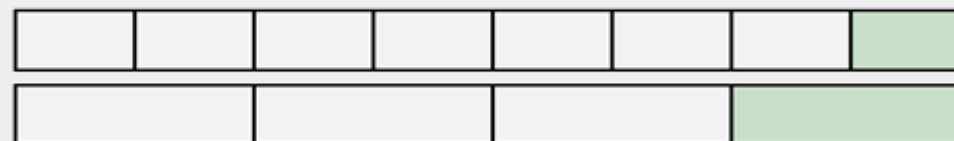
Floating point vectors:

- 8-way single
- 4-way double



Floating point scalars:

- single
- double

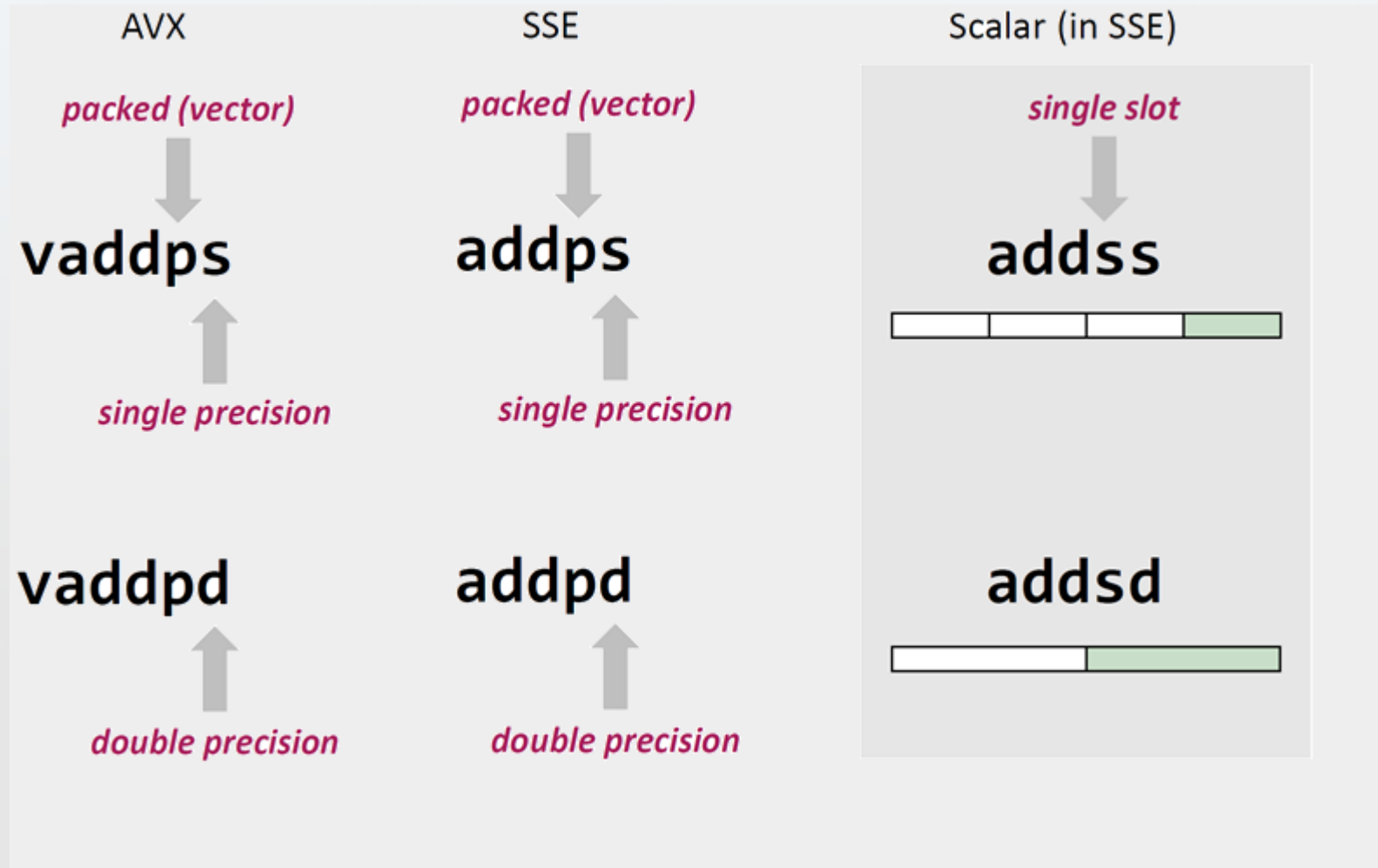


CONVENIO

TEMA 5: LAB PARALELISMO DE DATOS - SIMD

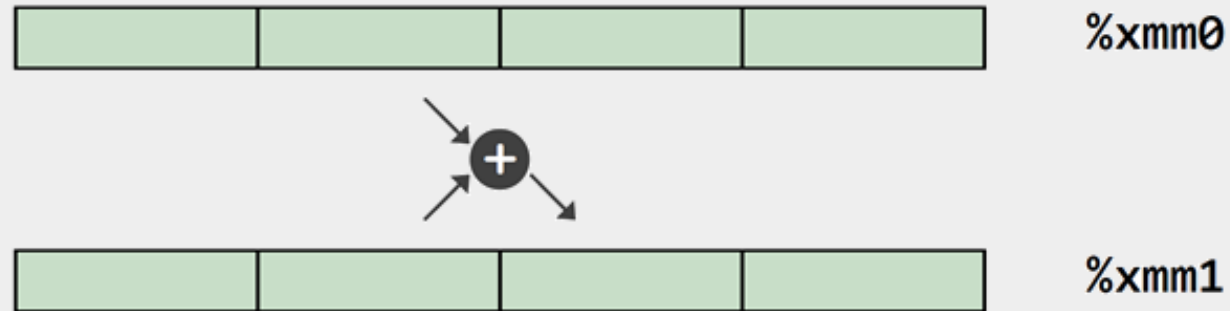


NOMBRES INSTRUCCIONES (ENSAMBLADOR)



INSTRUCCIONES SSE3/SSE4: EJEMPLOS

Single precision *4-way vector add*: `addps %xmm0, %xmm1`



Single precision *scalar add*: `addss %xmm0, %xmm1`

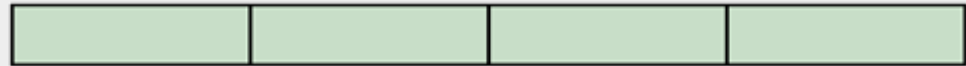


INSTRUCCIONES AVX: EJEMPLOS

Double precision *4-way vector add*: `vaddpd %ymm1 %ymm0 %ymm1`



%ymm0



%ymm1

(two-operand!)

Double precision *scalar add (in SSE2)*: `addsd %xmm0 %xmm1`



%xmm0



%xmm1

SSE/AVX INTRINSICOS

- Funciones C codificadas en ensamblador
 - Cada función representa una instrucción SIMD (o un pequeño conjunto de instrucciones)
 - Es como escribir ensamblador dentro de código C
- Un código C que incluya intrínsecos SIMD debe añadir un archivo de cabecera específico:
 - SSE4: `#include <smmintrin.h>`
 - AVX/AVX512: `#include <immintrin.h>`
- Los archivos de cabecera deben incluirse en el programa para indicar al compilador cómo asignar intrínsecos a instrucciones de máquina GNU CC (GCC)
 - El compilador GCC habilita el soporte intrínseco con las opciones `"-msse4"` o `"-mavx2"`
 - Ejemplos:
 - `gcc -O2 -msse4 code.c -o code` (soporte SSE4)
 - `gcc -O2 -mavx2 code.c -o code` (compatibilidad con AVX)



TIPO DE DATOS

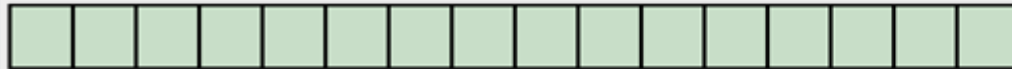
TEMA 5: LAB PARALELISMO DE DATOS - SIMD



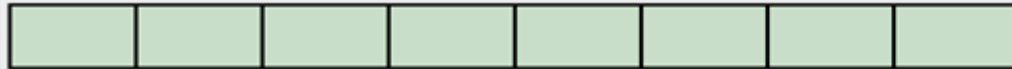
SSE INTRINSICS: DATA TYPES

- Tipos de datos

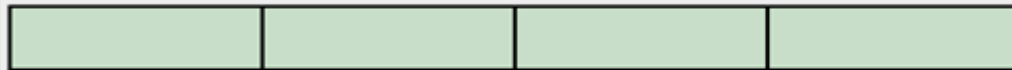
```
__m128  f;    // = {float f0, f1, f2, f3}  
__m128d d;    // = {double d0, d1}  
__m128i i;    // 16 8-bit, 8 16-bit, 4 32-bit, or 2 64-bit ints
```



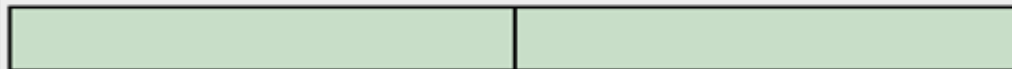
ints



ints



ints or floats

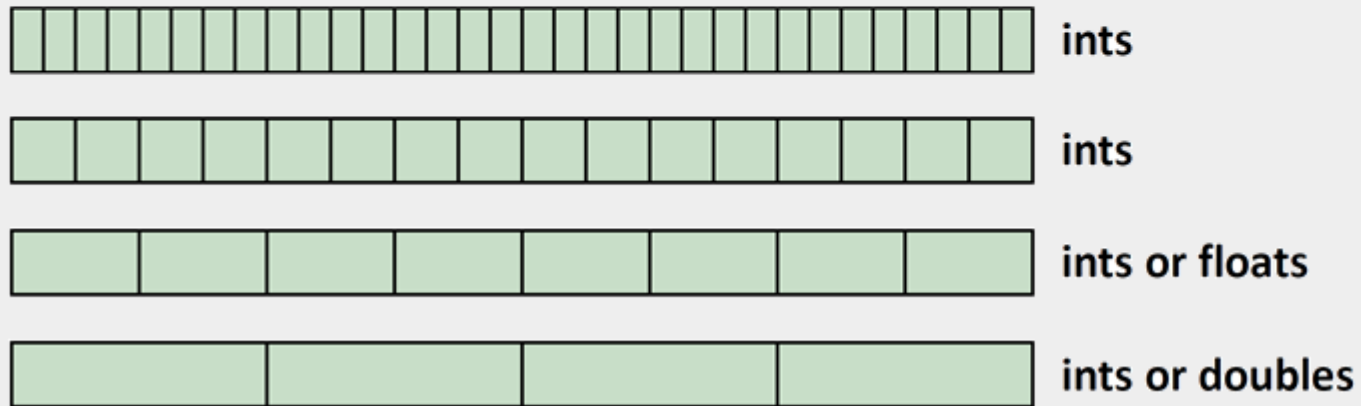


ints or doubles

AVX INTRINSICS: DATA TYPES

- Tipos de datos

```
__m256 f;    // = {float f0, f1, f2, f3, f4, f5, f6, f7}  
__m256d d;    // = {double d0, d1, d3, d4}  
__m256i i;    // 32 8-bit, 16 16-bit, 8 32-bit, or 4 64-bit
```



AVX512 INTRINSICOS: DATA TYPES

- Tipos de datos

```
__m512 ;  
__m512d ;  
__m512i ;
```

```
// puede contener 16 float (32-bit cada uno)  
// puede contener 8 double (64-bit cada uno)  
// 64 8-bit, 32 16-bit, 16 32-bit o 8 64-bit
```



INSTRUCCIONES

TEMA 5: LAB PARALELISMO DE DATOS - SIMD



SSE/AVX INTRINSICOS: INSTRUCCIONES

- Instrucciones nativas (mapea una-a-una con ensamblador)
 - `_mm_load_ps()` ↔ `movaps`
 - `_mm_add_ps()` ↔ `addps`
 - `_mm_mul_pd()` ↔ `mulpd`
 - `_mm256_load_pd()` ↔ `vmovapd`
 - `_mm256_add_pd()` ↔ `vaddpd`
 - `_mm256_mul_pd()` ↔ `vmulpd`
 - ...
- Multi instrucciones (mapea a varias instrucciones ensamblador)
 - `_mm_set_ps()`, `_mm_set1_ps()`, ...
 - `_mm256_set_pd()`, `_mm256_set1_pd()`, ...
- Macros y helpers
 - `_MM_TRANSPOSE4_PS()`
 - `_MM_SHUFFLE()`
 - ...



SSE/AVX INTRINSICOS: INSTRUCCIONES

- Instrucciones

- Nomenclatura (convenio):

- SSE: `_mm_<intrin_op>_<suffix>`
 - AVX: `_mm256_<intrin_op>_<suffix>`

- Ejemplo:

```
// a[] must be 16-byte aligned  
float a[4] = {1.0, 2.0, 3.0, 4.0};  
__m128 t = _mm_load_ps(a[0]);
```

p: packed (vector)
s: single precision

○ Resultado: t:

1.0	2.0	3.0	4.0
-----	-----	-----	-----

- Ejemplo:

```
// a[] must be 32-byte aligned  
double a[4] = {1.0, 2.0, 3.0, 4.0};  
__m256d t = _mm256_load_pd(a[0]);
```

p: packed (vector)
d: double precision

○ Resultado: t:

1.0	2.0	3.0	4.0
-----	-----	-----	-----

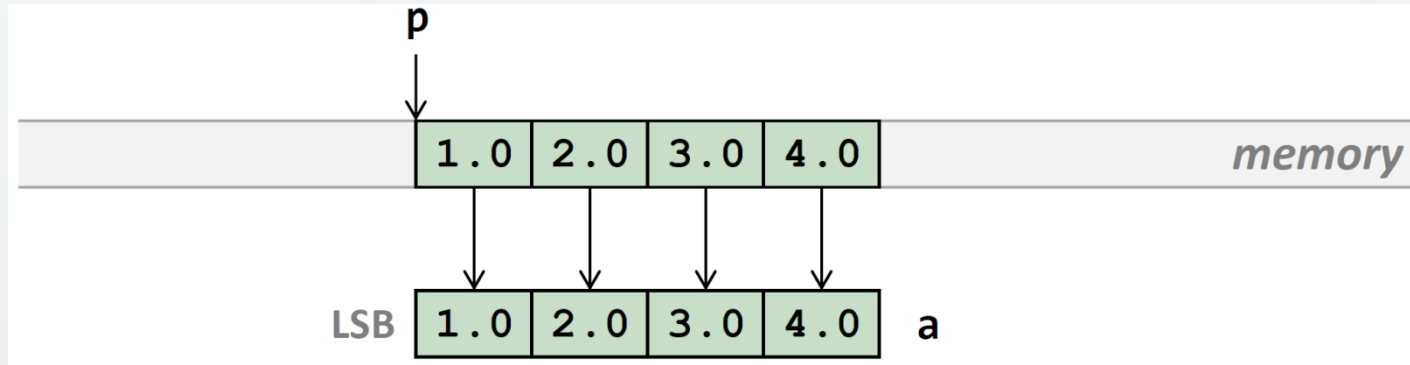


CATEGORIAS DE INTRINSECOS

- Load and store (loads and stores analogous)
- Constants
- Arithmetic
- Comparison
- Conversion
- Shuffles
- https://software.intel.com/sites/landingpage/IntrinsicsGuide/#techs=SSE,SSE2,SSE3,SSSE3,SSE4_1,SSE4_2,AVX&expand=451



INTRINSECOS: LOADS



```
__m128 a = _mm_load_ps(p) // p 16-byte aligned
```

```
__m128 a = _mm_loadu_ps(p) // p not aligned
```

It might be expensive

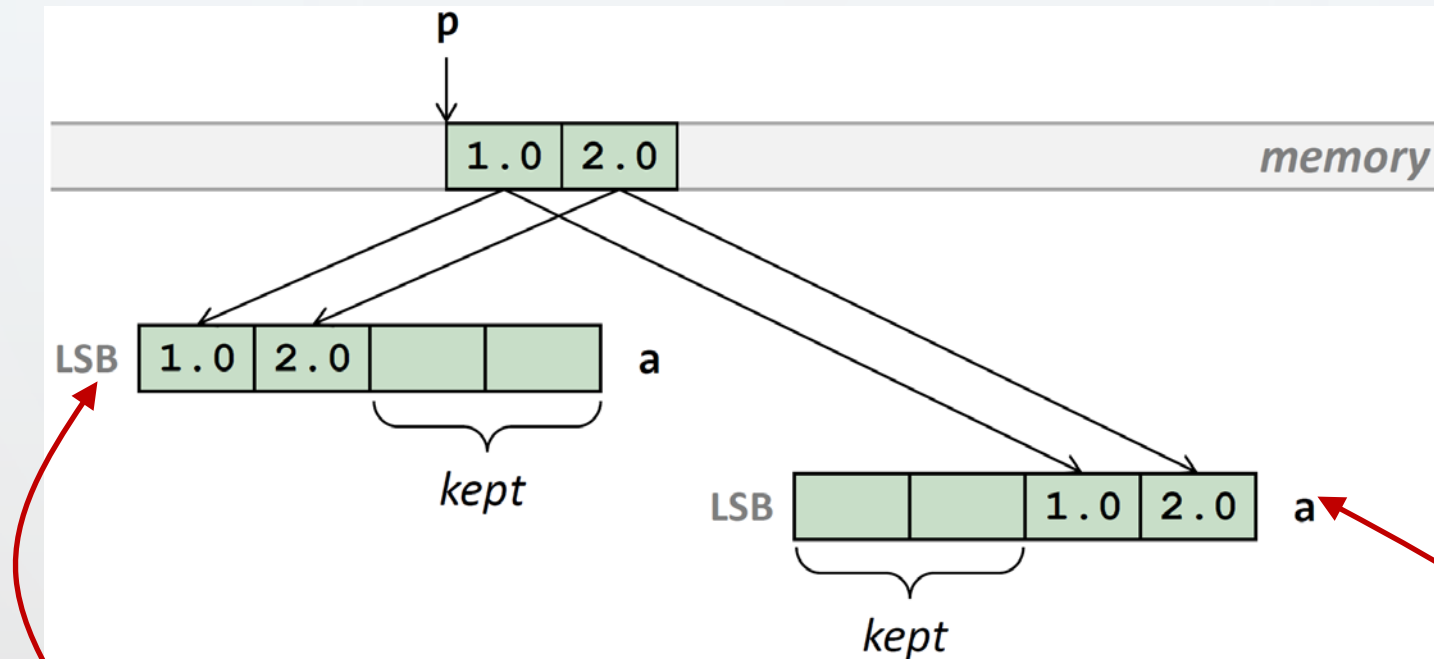
```
__m256d a = _mm256_load_pd(p) // p 32-byte aligned
```

```
__m256d a = _mm256_loadu_pd(p) // p not aligned
```

It might be expensive



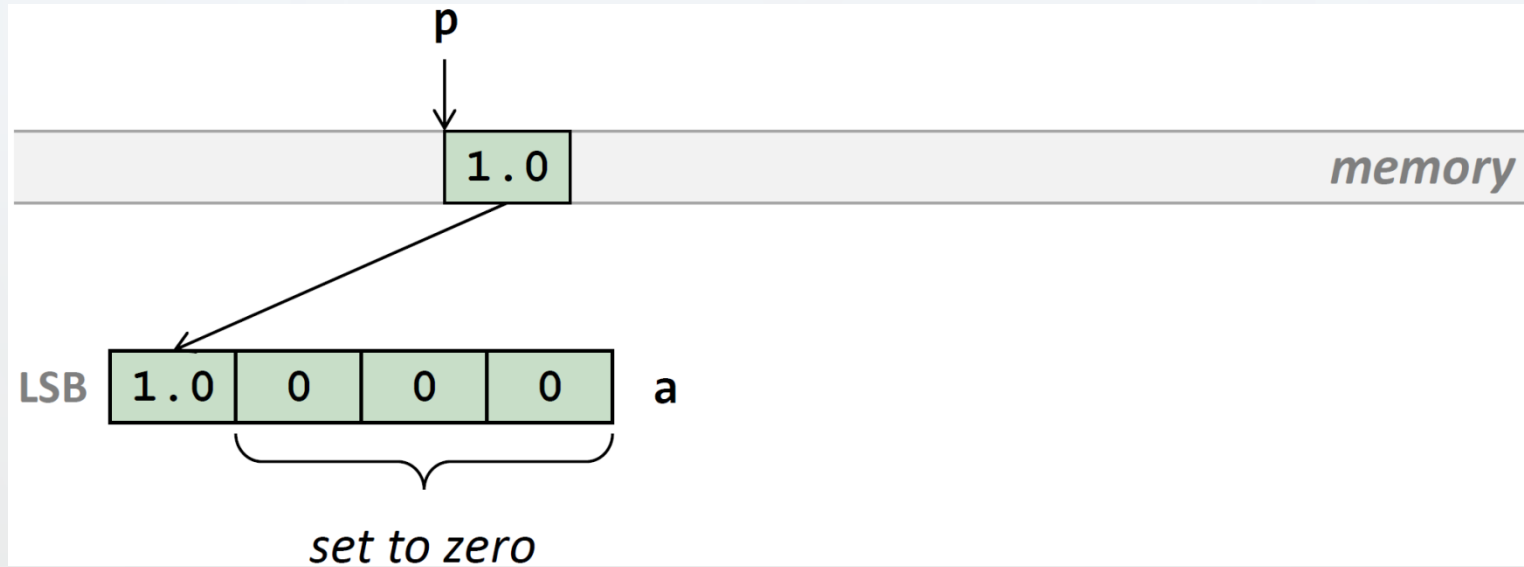
INTRINSECOS: LOADS



```
__m128 a = _mm_loadl_pi(a,p) // p 8-byte aligned
```

```
__m128 a = _mm_loadh_pi(a,p) // p 8-byte aligned
```


INTRINSECOS: LOADS



```
__m128 a = _mm_load_ss(p) // p any alignment
```

INTRINSECOS: CTES

LSB

1.0	2.0	3.0	4.0
-----	-----	-----	-----

 a

```
__m128 a = _mm_set_ps(4.0,3.0,2.0,1.0)
```

```
__m256d a = _mm256_set_pd(4.0,3.0,2.0,1.0)
```

LSB

1.0	1.0	1.0	1.0
-----	-----	-----	-----

 b

```
__m128 b = _mm_set1_ps(1.0)
```

```
__m256d b = _mm256_set1_pd(1.0)
```

LSB

1.0	0	0	0
-----	---	---	---

 c

```
__m128 c = _mm_set_ss(1.0)
```

LSB

0	0	0	0
---	---	---	---

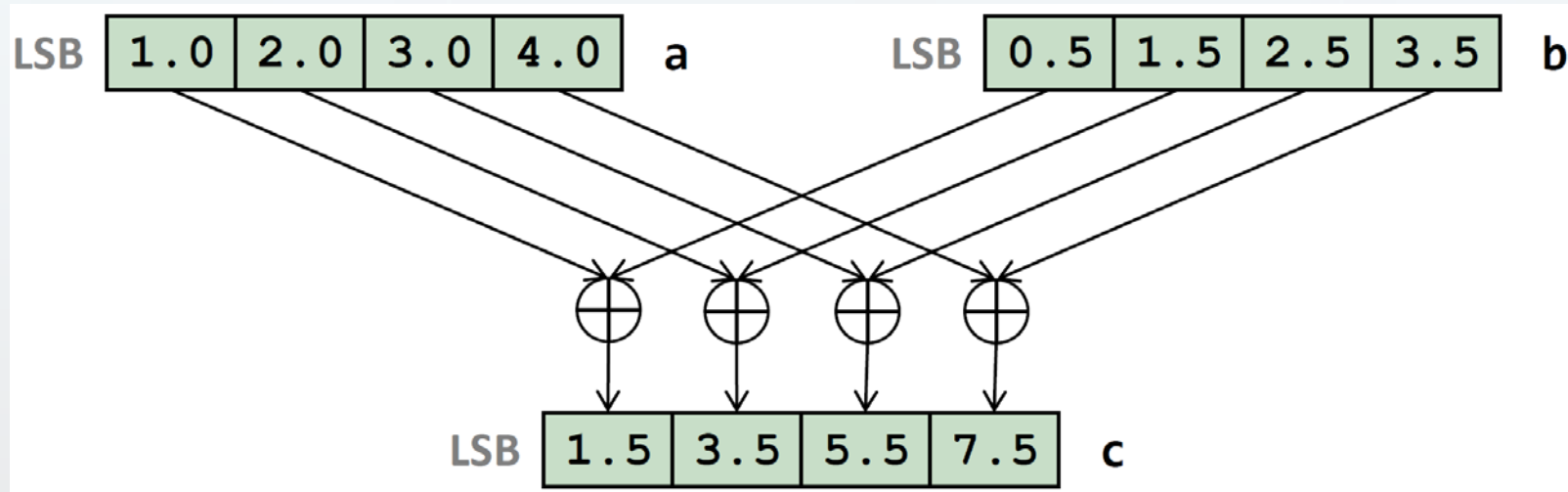
 d

```
__m128 d = _mm_setzero_ps()
```

```
__m256d d = _mm256_setzero_pd()
```



INTRINSECOS: ARITMÉTICAS



`__m128 c = _mm_add_ps(a, b)`

`__m256d c = _mm256_add_pd(a, b)`

O Analogous:

`__m128 c = _mm_sub_ps(a, b)`

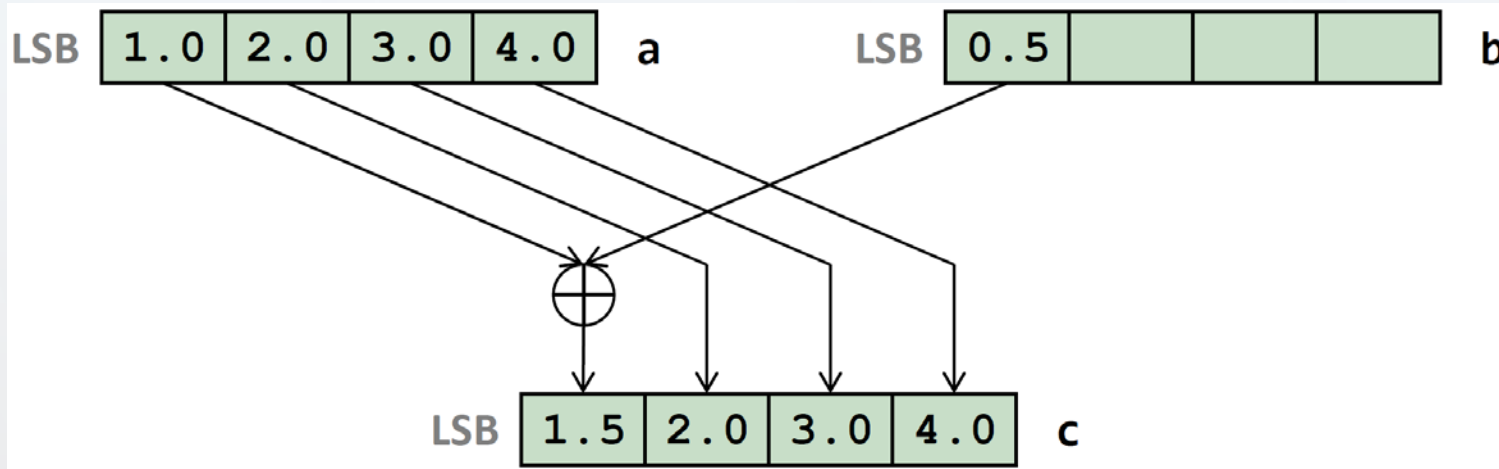
`__m256d c = _mm256_sub_pd(a, b)`

`__m128 c = _mm_mul_ps(a, b)`

`__m256d c = _mm256_mul_pd(a, b)`

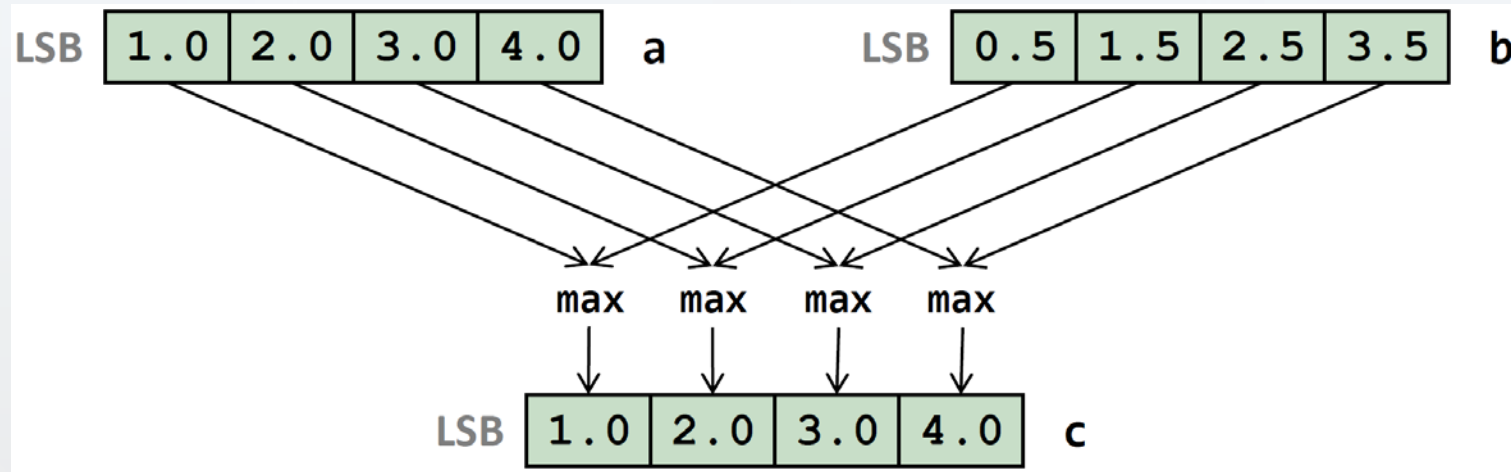


INTRINSECOS: ARITMÉTICAS



```
__m128 c = _mm_add_ss(a, b)
```

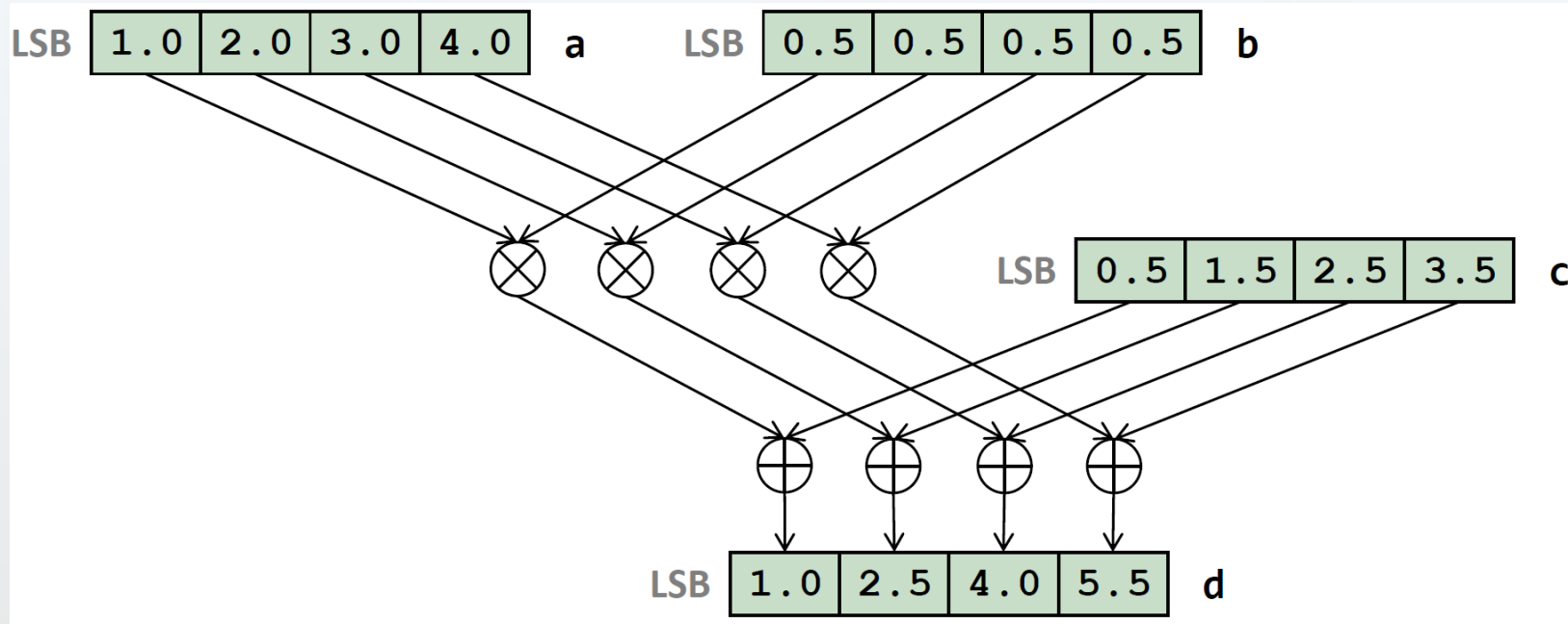
INTRINSECOS: ARITMÉTICAS



```
__m128 c = _mm_max_ps(a, b)
```

```
__m256d c = _mm256_max_pd(a, b)
```

INTRINSECOS: ARITMÉTICAS



```
__m256d d = _mm256_fmadd_pd(a, b, c)
```

O Analogous:

```
__m256d d = _mm256_fmsub_pd(a, b, c)
```

CUESTIONES A TENER EN CUENTA

TEMA 5: LAB PARALELISMO DE DATOS - SIMD



SSE/AVX INTRINSICOS: CUESTIONES PRINCIPALES

- Alineación es importante

- SSE: alinea las variables a 16 bytes (=128 bits)
- AVX: alinea las variables a 32 bytes (=256 bits)
- AVX512: alinea las variables a 64 bytes (=512 bits)

- Ejemplos:

- Sin alinear: `float A[N]`
- Alineado a 16 bytes: `float A[N] __attribute__((aligned(16)))`
- No alineado: `A = (float *) malloc(N*sizeof(float *))`
- Alineado a 16 bytes: `A = (float *) malloc(N*sizeof(float *))`
`posix_memalign((void *)&A,16,N*sizeof(float *))`

- Es necesario realizar loads y stores explícitamente

- Mezclar Tecnologías (SSE y AVX p.ej.) puede acarrear penalizaciones

